

Overview

🎯 In this course, you will learn the key concepts of data-driven testing in Squish. Additionally, you will learn to make a test case data-driven.

What Will I Learn ?

1. What is data-driven testing, and what are its advantages
2. What is the format of test data, and how to create test data for data-driven testing in Squish.
3. How to make a test case data-driven.
4. What is GUI coverage, and when is it used.

[GitHub - qt-learning/Data-Driven-Testing · GitHub](#)

[Squish API | Squish 9.2](#)

[Squish for Qt Tutorials | Squish 9.2](#)

[uibrowser | Squish 9.2](#)

Introduction

Data-driven testing is a software testing methodology where test input and output data is not written in the test script but stored in an external file that is loaded at runtime. This allows for the separation of test data and test script code when testing an AUT.

There are many advantages to using data-driven testing, including the ability to modify test values in the test data, rather than the test script. This means it is not required to include hard-coded values in the script itself, since data can be stored in the separate test data file or database.

With the data in a separate file or database, it is easier to extend tests by adding more values to the test data. Data-driven testing can be used to create a clear separation of responsibilities for test creation between programmers and test engineers who may not be experts in how the script works.

Squish supports data-driven testing with features to convert test scripts to data-driven code and an API for handling test data.

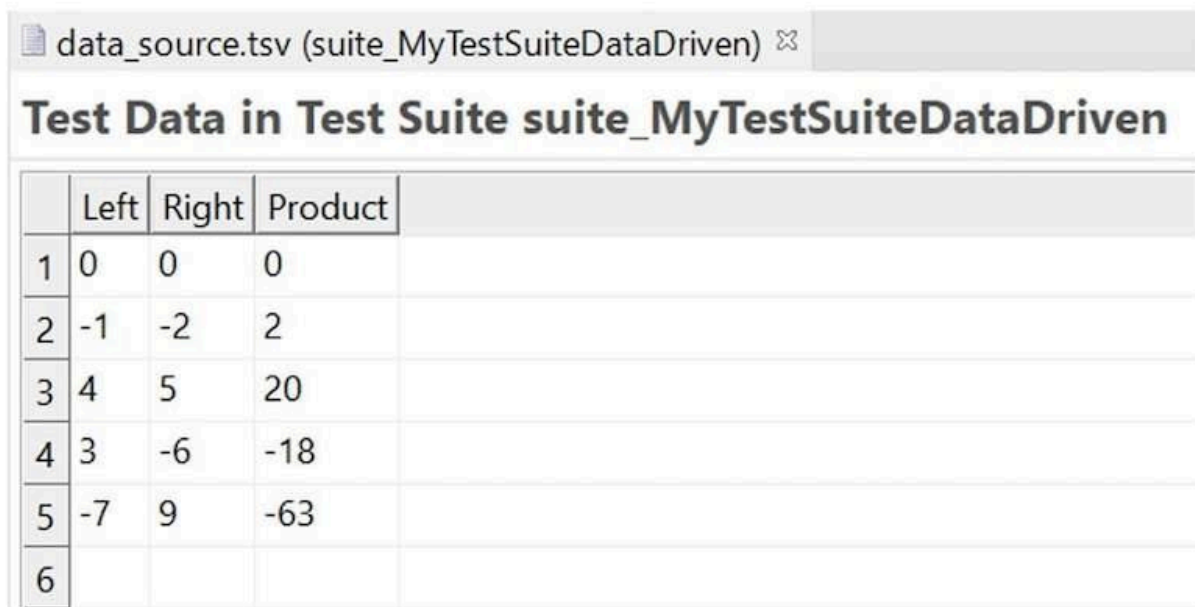
Test Data

When creating data-driven tests, test data stored in a table format can be used to store the input and expected output. Squish supports the following table-based data sources:

- Excel (.xlsx)
- CSV*
- TSV*

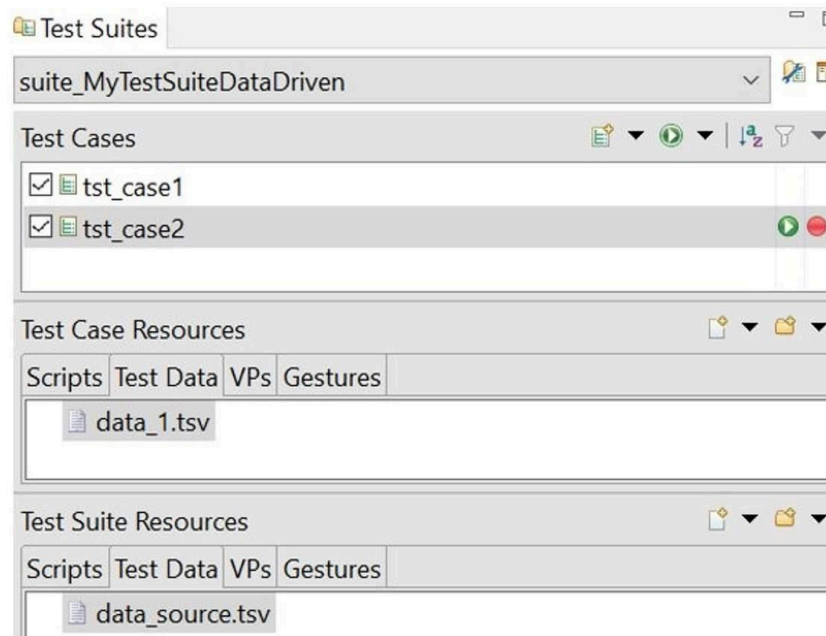
* .csv and .tsv files are assumed to be in the Unicode UTF-8 encoding

It is possible to create data tables using the Squish table editor by accessing Test [Case/Suite] Resources > Test Data > New Test Resource. Tables can also be created externally and imported into the Squish IDE using the [Import Squish Resource](#) dialog accessed at File > Import Test Resource.



	Left	Right	Product
1	0	0	0
2	-1	-2	2
3	4	5	20
4	3	-6	-18
5	-7	9	-63
6			

Test data can be stored in Test Case Resources, Test Suite Resources, or in a custom directory.



The return value of the [findFile\(where, filename\)](#) function can be used to access test data within a test script. This function takes two arguments and returns the path of a given filename. When using data stored in the Test [Case/Suite] Resources, the first argument of the findFile(where, filename) function can be the keyword “testdata”, which will cause findFile(where, filename) to look first in the “testdata” directory of the test case for the data file. If findFile(where, filename) does not locate a test data file in this directory, it will look in the “testdata” directory of the test suite. Lastly, findFile(where, filename) will try looking in the testcase’s directory, and if no file is found it will return a [LookupError](#).

Python

```
file_path = findFile("testdata", "data_source.tsv")
```

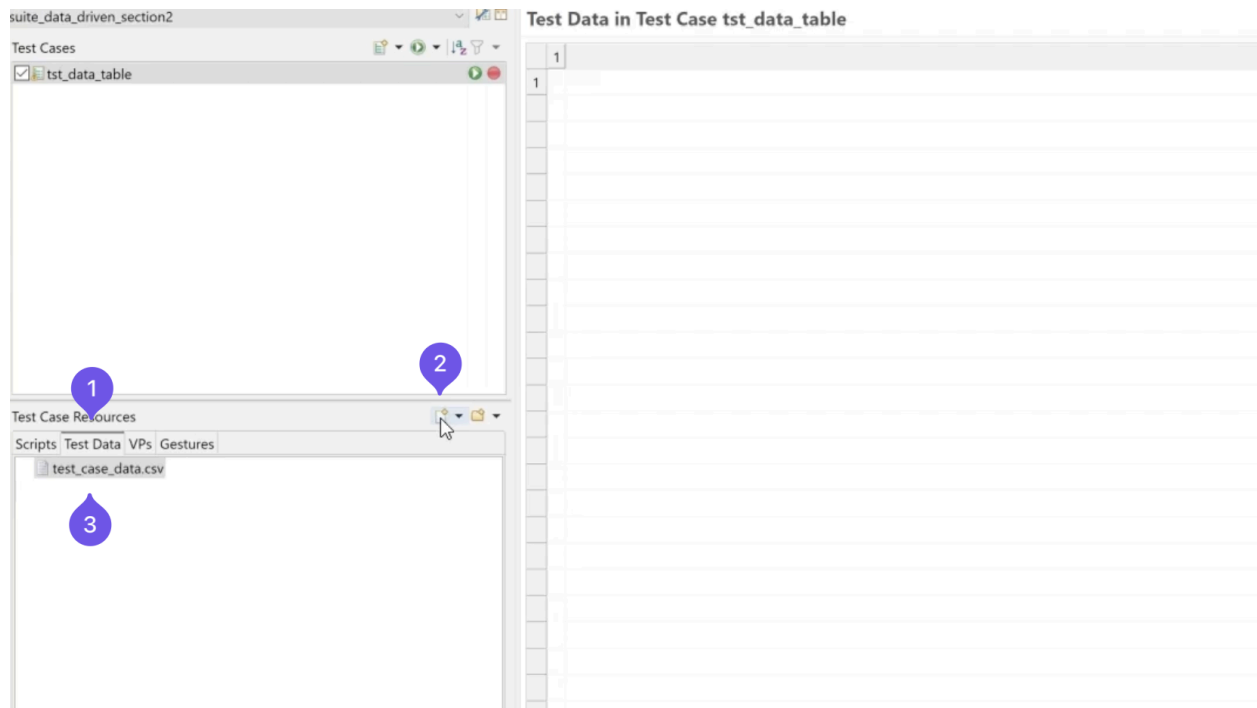
STEPS - findFile() Examples With Various Data Sources

STEP 00 = Starting Point

In this demo, we went over the steps of creating test data in the Squish IDE, as well as how to access test data in a test script.

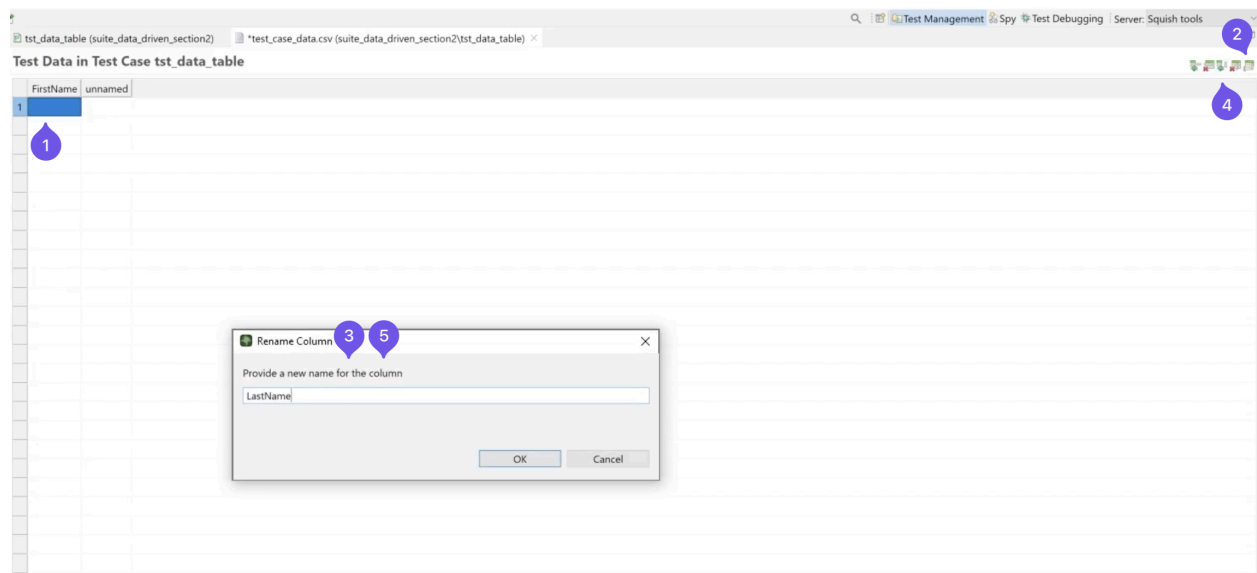
To begin, we had a test suite open in the Squish IDE with a test case called `tst_data_table`. The test suite was not configured to target any AUT.

STEP 01 = Creating data to be stored in our test case resources



1. We selected the Test Data tab in "Test Case Resources"
2. and the New Test Resource icon. This created a .tsv file by default called "data_one.tsv,"
3. We renamed the file test_case_data.csv.

STEP 02 = Setting up the table in Test Data view



After renaming our file, the test data file opened in the Squish IDE, and the Test Data view was displayed. This view showed a populatable table along with editable row and column names. We set up the table to have some column names.

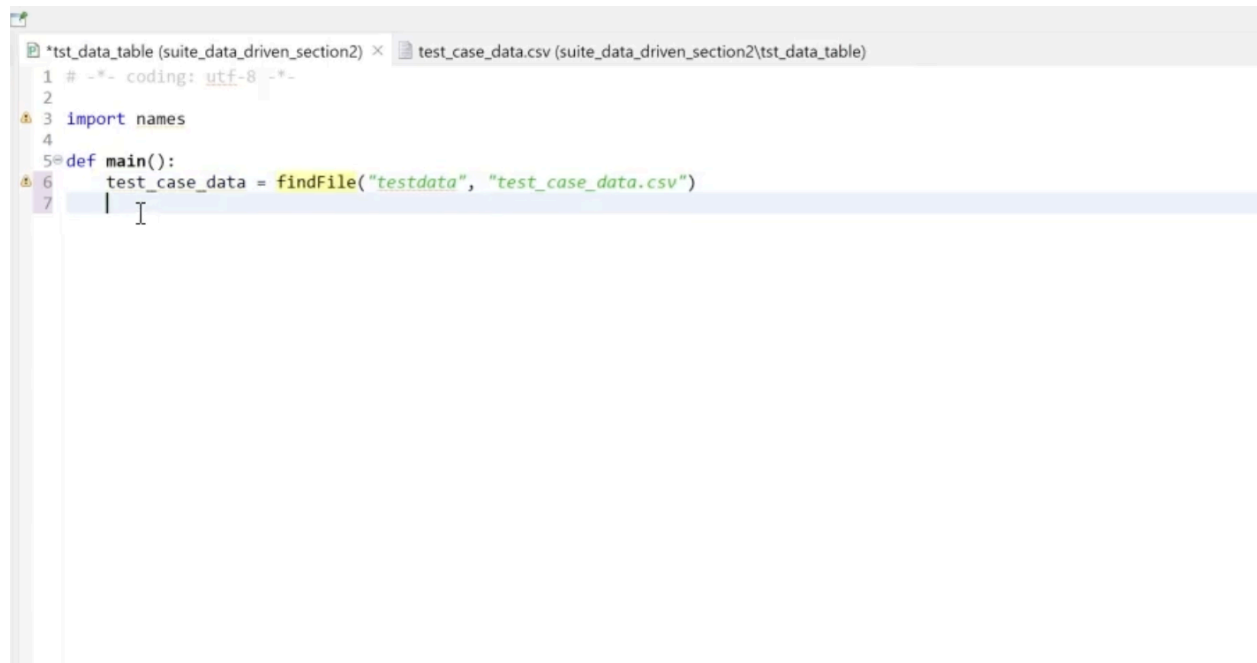
1. We clicked on the first cell within the table.
2. We selected the Rename Column icon on the far right.
3. We renamed the column "FirstName".
4. After this, we added another column to our table by selecting the Insert Column icon.
5. We renamed the second column "LastName".

STEP 03 = Populating the table

[illegible]

With two columns in our table, we could begin to populate the table with some first and last names. We added two entries for John and Jane Doe. We did not need to insert or rename rows manually since new numbered rows were automatically added to the table with new entries.

STEP 04 = Identifying and accessing test data resources



```
*tst_data_table (suite_data_driven_section2) × test_case_data.csv (suite_data_driven_section2\tst_data_table)
1 # -*- coding: utf-8 -*-
2
3 import names
4
5 def main():
6     test_case_data = findFile("testdata", "test_case_data.csv")
7     |
```

Next, we went back to the test script for our test case. In our test script, we wanted to locate our test data files and log the path at which they are stored to demonstrate how to access the files. To do this, we utilized the commonly used function, `findFile(where, filename)`

FindFile(where, filename) function looks through the test case resources, then the test suite resources, and returns a path to the first file which matches the name of the file we are looking for.

The function takes **two arguments**, the first of which is a string keyword, to specify if we are looking in the scripts or test data directories. We added the first argument of "test data". The second argument is simply the name of the file. We added the **name of our file** to our `findFile(where, filename)` function arguments.

STEP 05 = Storing the findFile function's return value in a variable and logging the file path

```
tst_data_table (suite_data_driven_section2) × test_case_data.csv (suite_data_driven_section2\tst_data_table)
1 # -*- coding: utf-8 -*-
2
3 import names
4
5 def main():
6     test_case_data = findFile("testdata", "test_case_data.csv")
7
8     test.log(test_case_data)
9     |
```

The findFile function returns a path to our test data file within our test case resources. We saved that return value in a variable:

test_case_data = findFile("testdata", "test_case_data.csv")

Now, we could log that file path to show we have correctly found our test data file:

test.log(test_case_data)

STEP 06 = Running the test case

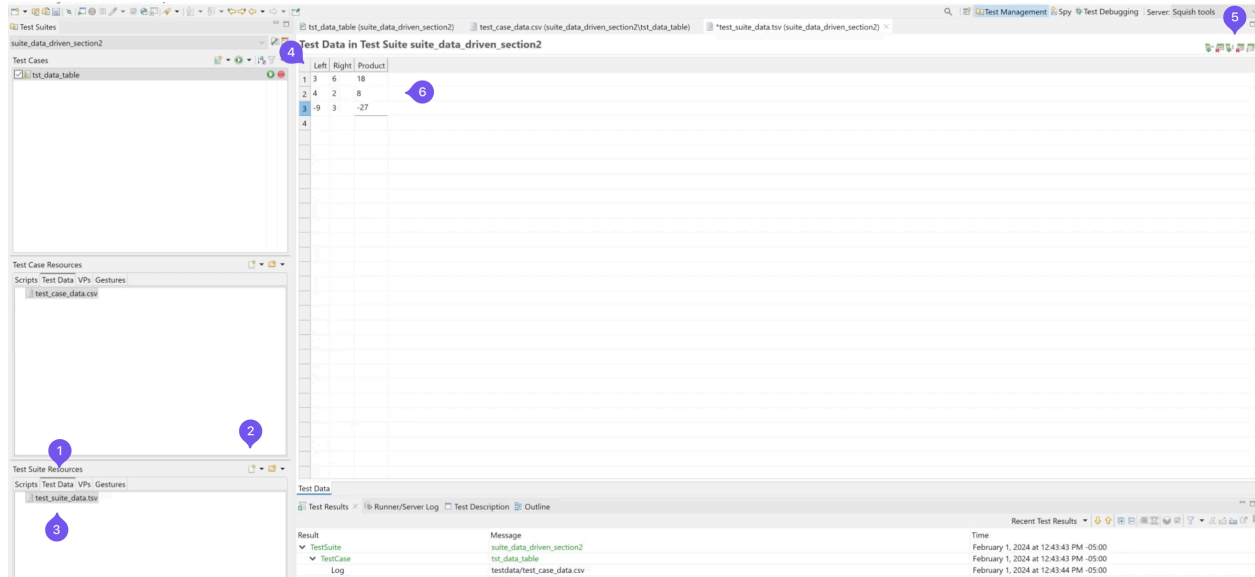


The screenshot shows the 'Test Results' window with a table of results. The table has three columns: 'Result', 'Message', and 'Time'. The 'Result' column shows a hierarchy: 'TestSuite' expanded to 'TestCase', which is further expanded to 'Log'. The 'Message' column shows the path 'testdata/test_case_data.csv'. The 'Time' column shows the timestamp 'February 1, 2024 at 12:43:44 PM -05:00'.

Result	Message	Time
TestSuite	suite_data_driven_section2	February 1, 2024 at 12:43:43 PM -05:00
TestCase	tst_data_table	February 1, 2024 at 12:43:43 PM -05:00
Log	testdata/test_case_data.csv	February 1, 2024 at 12:43:44 PM -05:00

Next, we run our test case. We saw upon running the test, in our Test Results summary, we had logged the file path for our test data resource.

STEP 07 = Create data to be stored in Test Suite Resources



Similarly to what we did previously, we created a new Test Suite Resource by:

1. Selecting the Test Data tab on "Rest Suite Resources".
2. Clicking the New Test Resource icon in our Test Suite Resources.
3. We renamed the file "test_suite_data.tsv"

This opened the Test Data view, where we could add our test data to the file. In this test data file, we added some data pertaining to some calculations, with two columns containing numbers that would be multiplied and a third column containing an expected product.

We added the columns like previously:

4. Clicking on the first cell of the table.
5. Selecting the rename icon and renaming the column "Left". We created two more columns and named them "Right" and "Product".
6. Lastly, we added three rows of data.

STEP 08 = Identifying the created file and logging the file to the test


```
tst_data_table (suite_data_driven_section2) × test_case_data.csv (suite_data_driven_section2\tst_data_table) test_suite_data.tsv (suite_data_driven_section2\tst_data_table)
1 # -*- coding: utf-8 -*-
2
3 import names
4
5 def main():
6     test_case_data = findFile("testdata", "test_case_data.csv")
7     test_suite_data = findFile("testdata", "test_suite_data.tsv")
8
9     test.log(test_case_data)
10    test.log(test_suite_data)
11
```

Next, we headed back to our test case and made sure we could identify the recently created file. We used the “find file” function with nearly the same syntax as earlier, the only difference was that we were looking for a file called “test suite data” now.

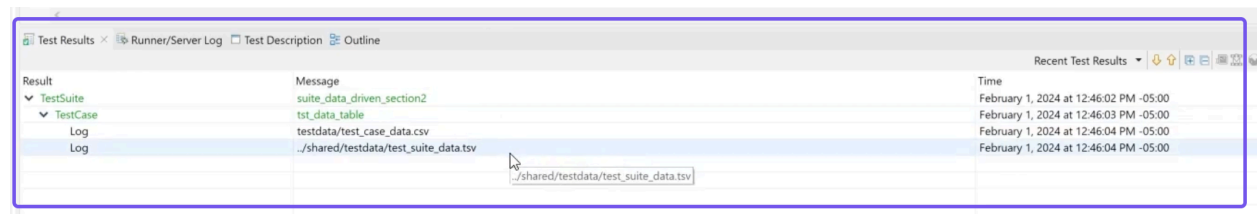
So we added:

test_suite_data = findFile(“testdata”, “test_suite_data.tsv”)

Now, we could log that file in our test to show we have the correct file path:

test.log(test_suite_data)

STEP 09 = Running the test case

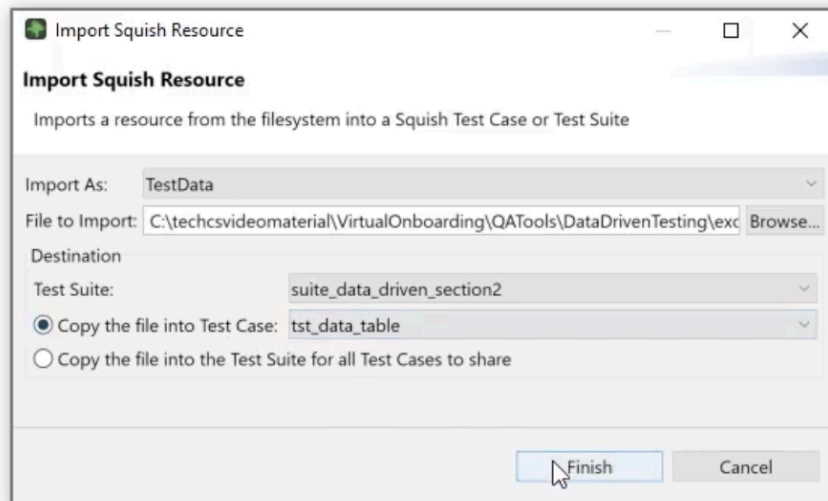


Result	Message	Time
TestSuite	suite_data_driven_section2	February 1, 2024 at 12:46:02 PM -05:00
Test Case	tst_data_table	February 1, 2024 at 12:46:03 PM -05:00
Log	testdata/test_case_data.csv	February 1, 2024 at 12:46:04 PM -05:00
Log	./shared/testdata/test_suite_data.tsv	February 1, 2024 at 12:46:04 PM -05:00

Running the test, we could see in our test results summary that the file in our Test Suite Resources was properly located.

STEP 10 = Importing resources to the Test Case or Test Suite Resources locations

```
8
9 test.log(test_case_data)
10 test.log(test_suite_data)
11
```



In addition to creating test data files within the Squish IDE, resources can be created outside the IDE and imported into a Test Case or Test Suite Resources location. To do this, we selected File > Import Test Resource.

This displayed the “Import Squish Resource” wizard from which we could select a few options. The first option allows us to specify what type of file we are importing. We selected “TestData.” Then we could specify the location of the file to import. Here we selected browse and chose the file called “excel data.xlsx,” which contained our test data. Lastly, we chose the destination of the test data file. For our example, we imported the Excel file as a Test Case Resource. We selected finish, upon which we could see our test data file added to our Test Case resources.

STEP 11 = Identifying the added file and logging the file to the test

```
1 # -*- coding: utf-8 -*-
2
3 import names
4
5 def main():
6     test_case_data = findFile("testdata", "test_case_data.csv")
7     test_suite_data = findFile("testdata", "test_suite_data.tsv")
8     excel_data = findFile("testdata", "excel_data.xlsx")
9
10    test.log(test_case_data)
11    test.log(test_suite_data)
12    test.log(excel_data)
13    |
```

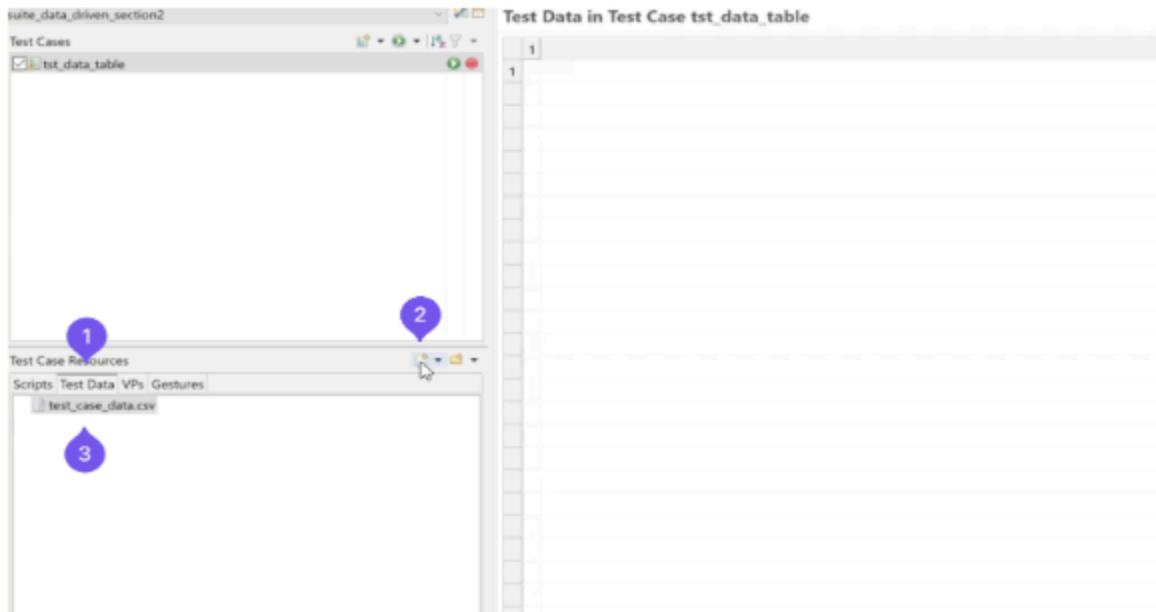
We added one more “find file” function to our code. We used the same syntax as the other two examples, but with the appropriate file name:

excel_data = findFile(“testdata”, “excel_data.xlsx”)

We also added a line to log the file location:

test.log(excel_data)

Creating data to be stored in our test case resources



1. We selected the Test Data tab in "Test Case Resources"
2. and the New Test Resource icon. This created a .tsv file by default called "data_one.tsv,"
3. We renamed the file test_case_data.csv.

```
Python  
file_path = findFile("testdata", "data_source.tsv")
```

tst_data_table (suite_data_driven_section2) × test_case_data.csv (suite_data_driven_section2)

```
1 # -*- coding: utf-8 -*-
2
3 import names
4
5 def main():
6     test_case_data = findFile("testdata", "test_case_data.csv")
7     test_suite_data = findFile("testdata", "test_suite_data.tsv")
8     excel_data = findFile("testdata", "excel_data.xlsx")
9
10    test.log(test_case_data)
11    test.log(test_suite_data)
12    test.log(excel_data)
13
```

Test Results × Runner/Server Log ☐ Test Description ☒ Outline

Result	Message
▼ TestSuite	suite_data_driven_section2
▼ TestCase	tst_data_table
Log	testdata/test_case_data.csv
Log	../shared/testdata/test_suite_data.tsv
Log	testdata/excel_data.xlsx

Test Data Functions

The [Squish API](#) contains [Test Data Functions](#) used to handle tabular data. The first of these functions is the [testData.dataset\(\)](#) function, which returns an array or a tuple of records from the data source. By default, `testData.dataset()` function can be given a file name as sole argument, as Squish will look in the test case's "testdata" directory first, followed by the test suite's shared "testdata" directory if the file is not found in the first directory. It is also possible to pass a filename that includes a path as the argument. `testData.dataset()` is often paired with the [findFile\(when, filename\)](#) function as it is possible to give `testData.dataset()` the return value of `findFile(when, filename)` as an argument.

Python

```
data_set = testData.dataset(findFile("testdata", "data_source.tsv"))
```

Once an array or tuple of records has been acquired, the [testData.field\(record, fieldName\)](#) function can be used to access the content of individual fields within the data table. The first argument in this function is the row to be accessed in the table. The second argument is the "fieldName," which is the name of the column in the table which should be accessed. The `testData.field(record, fieldName)` function is usually placed in a "for" loop to access each individual field within the data table.

Python

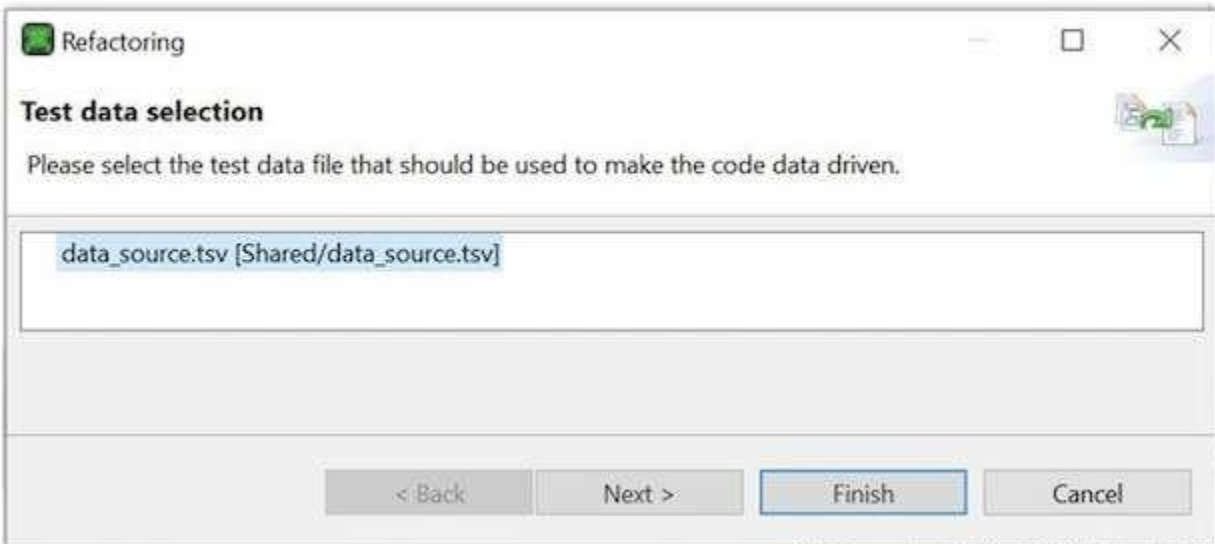
```
for record in data_set
    left = testData.field(record, "Left")
    right = testData.field(record, "Right")
    product = testData.field(record, "Product")
```

For more advanced test data functionality like creating or copying test data, see the following documentation: [Test Data Functions](#)

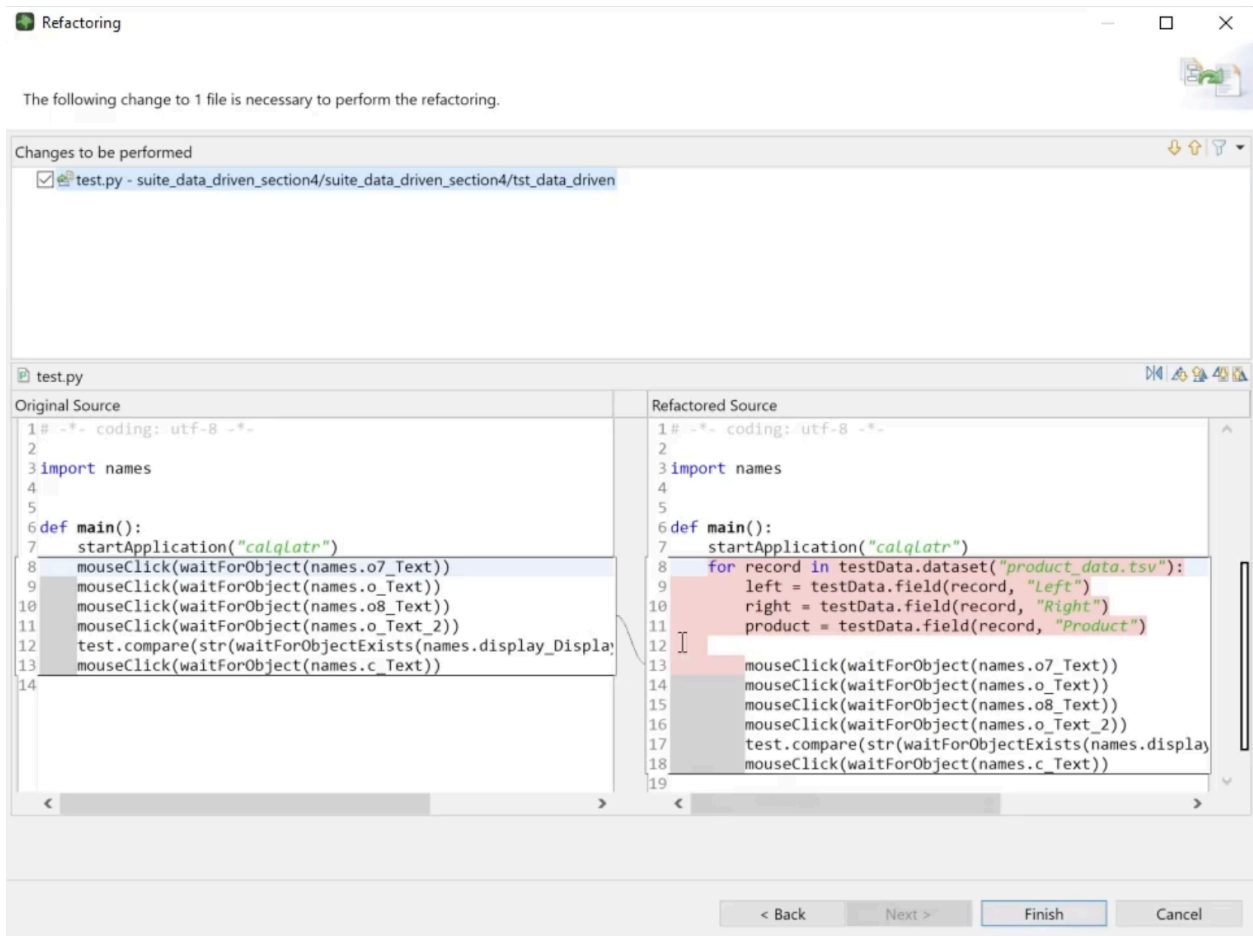
Making a Test Case Data-driven

The Squish IDE can convert code to data-driven code. Existing code can be made data-driven by selecting the appropriate lines, right-clicking, and selecting "Make Code Data Driven". The "Make Code Data Driven" option is only available if there is a test data file available to the test script.

Upon clicking "Make Code Data Driven", a refactoring wizard will appear. The wizard will first prompt the user to select some test data. Any test data sources available to the test script will appear in this menu.



By clicking the "Next" button, the refactoring wizard will show a preview of refactoring necessary to make the code data-driven. To create a data-driven test, the existing code will be placed in a for loop and some variables will be created based on the test data. The created variables will contain each column value from the table, and the "for" loop will iterate through each row, adjusting the variables accordingly.

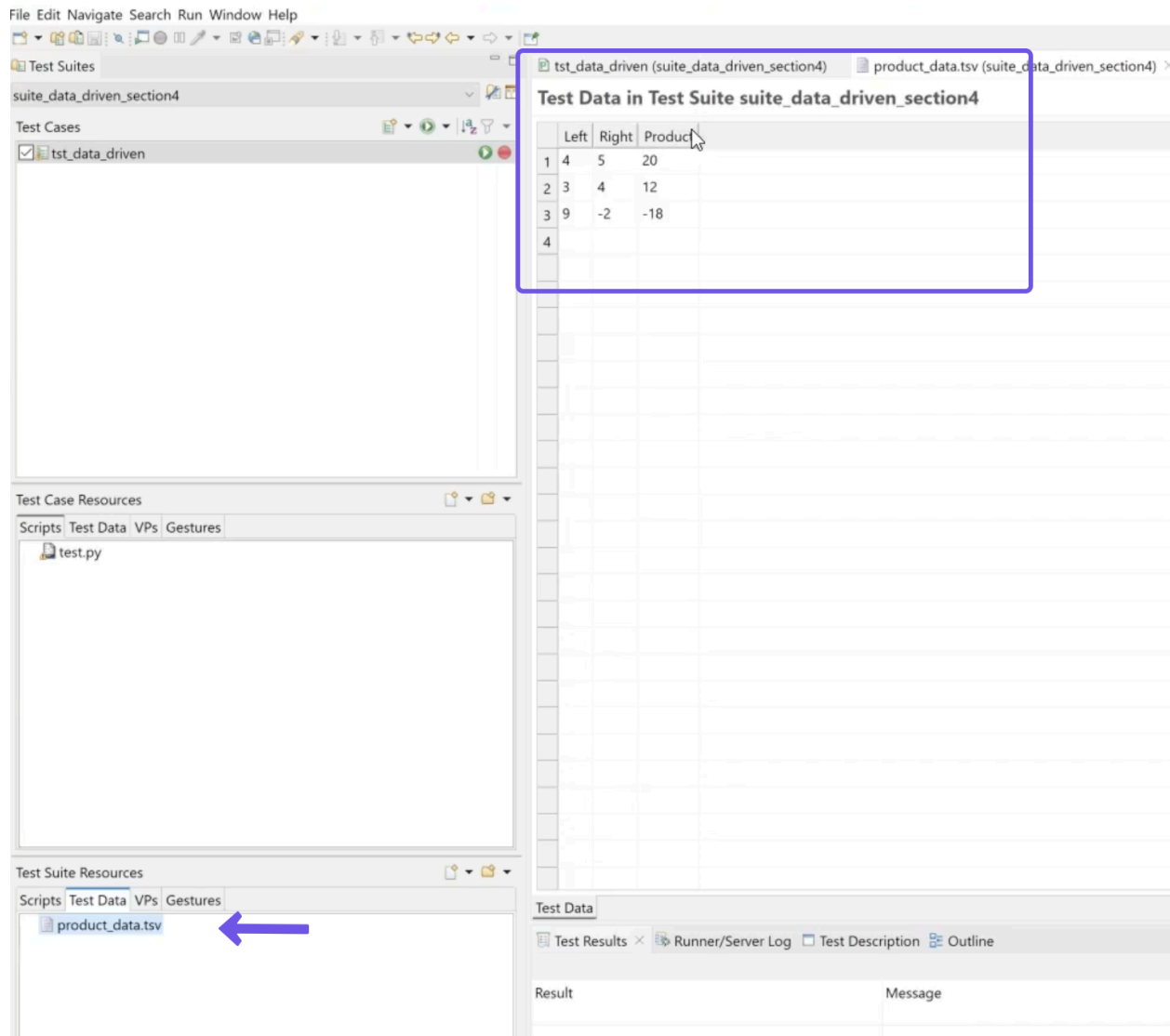


The “Finish” button can then be selected to make the changes in the test script. It is possible to undo changes made by the refactoring wizard. After the refactoring wizard is complete, the code is setup to be data-driven. Once these changes have been created, any hard coded values will still be present, so some refactoring will be necessary to replace the hard coded values with the newly created variables.


```
tst_data_driven (suite_data_driven_section4) × product_data.tsv (suite_data_driven_section4) names.py (suite_data_driven_section4)
1 # -*- coding: utf-8 -*-
2
3 import names
4
5 def enterValue(val):
6     flagNegative = False
7     if val[0] == '-':
8         flagNegative = True
9         val = val[1:]
10    for integer in val:
11        obj = {"container": names.o_QQuickView, "text": integer, "type": "Text", "unnamed": 1, "visible": True}
12        mouseClicked(waitForObject(obj))
13    if flagNegative:
14        mouseClicked(waitForObject(names.o_Text_3))
15
16 def main():
17     startApplication("calqlatr")
18     for record in testData.dataset("product_data.tsv"):
19         left = testData.field(record, "Left")
20         right = testData.field(record, "Right")
21         product = testData.field(record, "Product")
22
23         enterValue(left)
24         mouseClicked(waitForObject(names.o_Text))
25         enterValue(right)
26         mouseClicked(waitForObject(names.o_Text_2))
27         test.compare(str(waitForObjectExists(names.display_Display).displayedOperand), product)
28         mouseClicked(waitForObject(names.c_Text))
29
```

STEPS - How to Make a Test Data Driven With the Refactoring Wizard

STEP 00 = Starting point



In our demo, we went over the steps on how to create a data-driven test in Squish. As a starting point, we had a test suite with one test case in which we created a data-driven test. This test suite used the calculator example, which can be found in the examples subdirectory within the Squish package: **SQUISHDIR\examples\qt5\calculator\calculator.exe**

In the calculator example, we tested various products to ensure our calculator is working correctly. We had data provided by a test data file which specified the input and expected output to be used for this test. This test data file was already created and can be found in the test data directory within our Test Suite resources.

We started by taking a look at our test data by double-clicking the file in the Test Suite resources. In the test data, we had three columns, the first two being the left and right numbers

to be used in the multiplication test as inputs. The third column shows the products that we did a comparison to after the calculation.

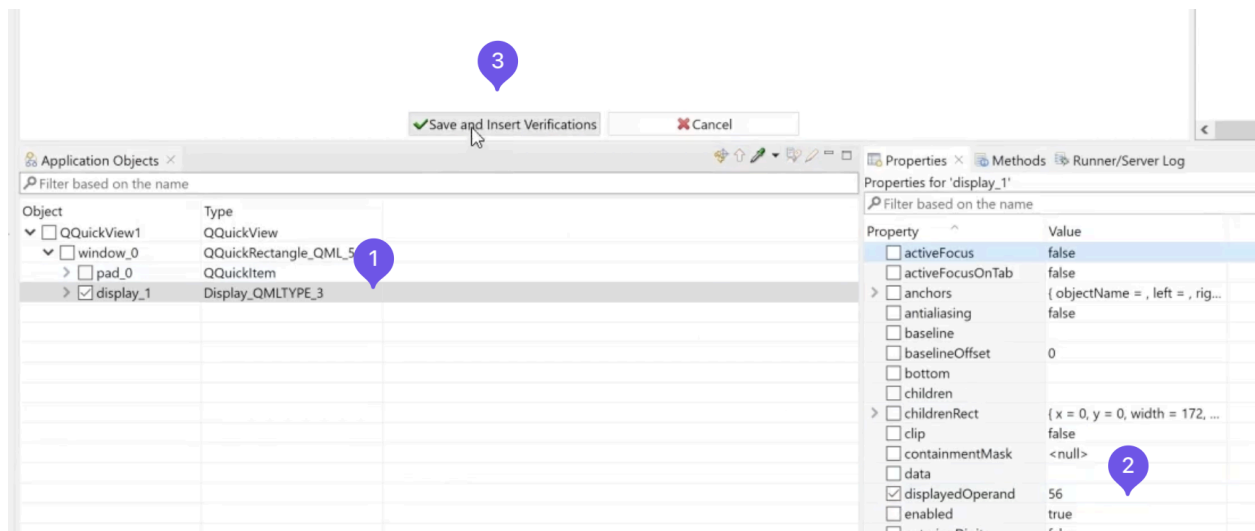
STEP 01 = Creating a base behaviour, part 1



We started by creating the base behavior for our test before making it data-driven. We simply recorded a multiplication operation by selecting the record button, which displayed the calculator.

1. We did some example multiplication in the calculator, selected 7 times 8, and clicked equals, which displayed a product.
2. Then, we added a verification point to verify this property value by selecting Verify > property in the Control Bar.

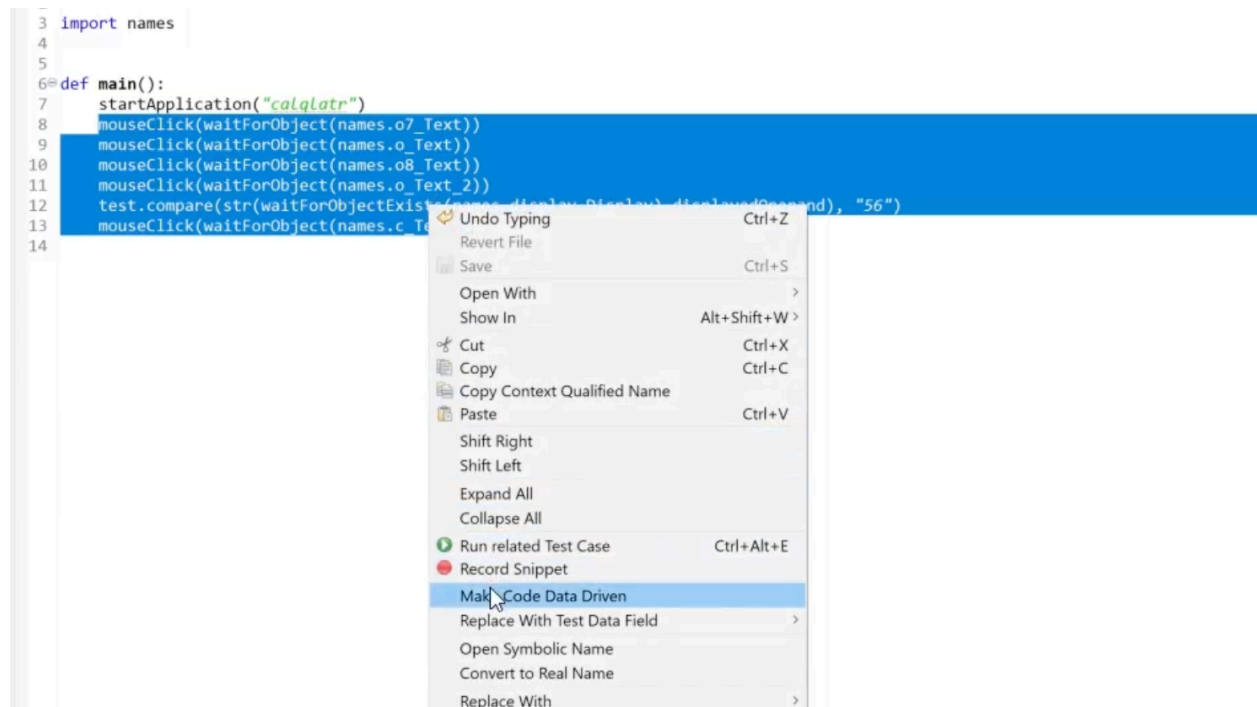
STEP 02 = Creating a base behaviour, part 2



1.-2. In the Application Objects view, we needed to select the appropriate object to do some property verification on. We selected the “display one” object and the “displayedOperand” property, which holds our result value.

3. In the Verification Point Creation view, we selected “Save and Insert Verifications.”
Next, we hit the clear button to set up future operations in our calculator and stopped recording.

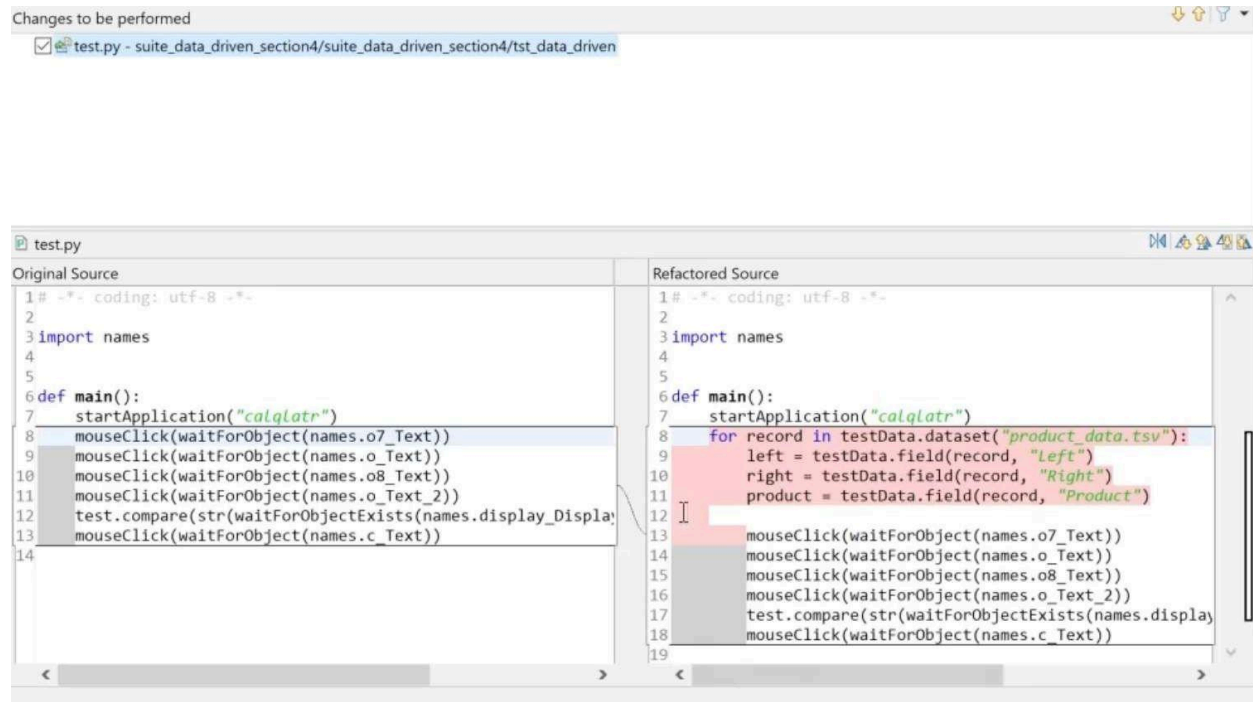
STEP 03 = Making the test data-driven, part 1



In the generated code, we noticed that our test script was using hard-coded values in the calculation. We wanted to extend this test by making it data-driven, so we could test any input to any expected output that appears in our test data.

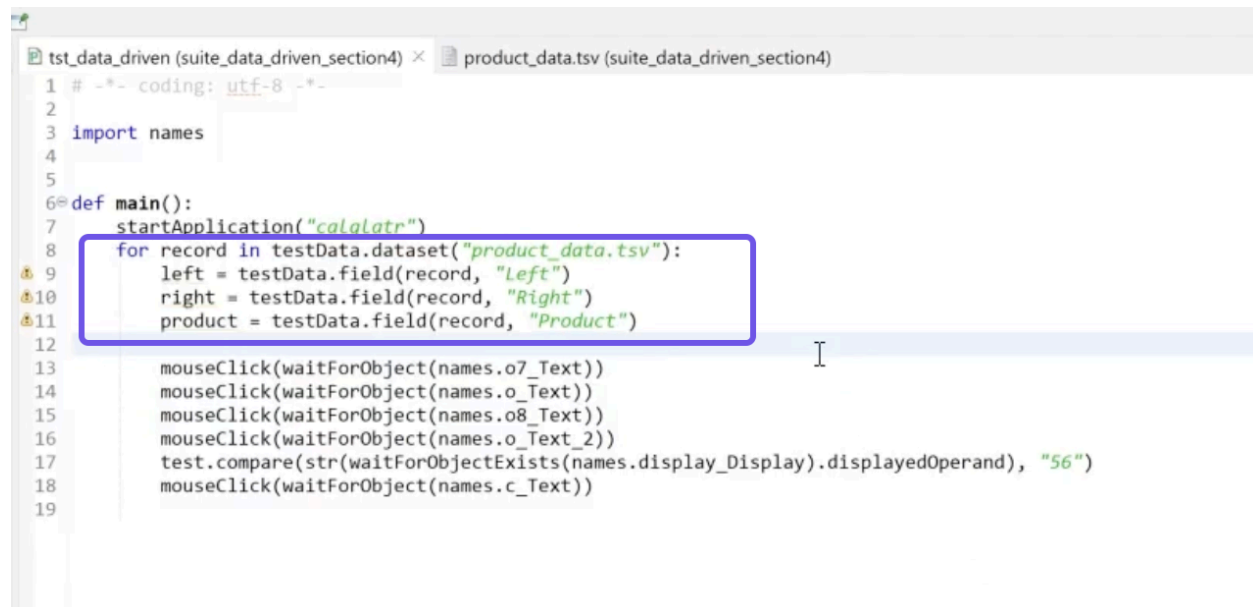
To do this, we highlighted the code that we wanted to make data-driven. Then, we right-clicked and selected “Make Code Data Driven.” This displayed the Refactoring wizard, which prompted us to select a test data file. We selected our product data file and clicked "Next".

STEP 04 = Making the test data-driven, part 2



The next window displayed all the changes that would be performed in a side-by-side comparison. We saw our original code on the left, and the refactored code on the right. We clicked finish and saw the refactored code appear in our test script.

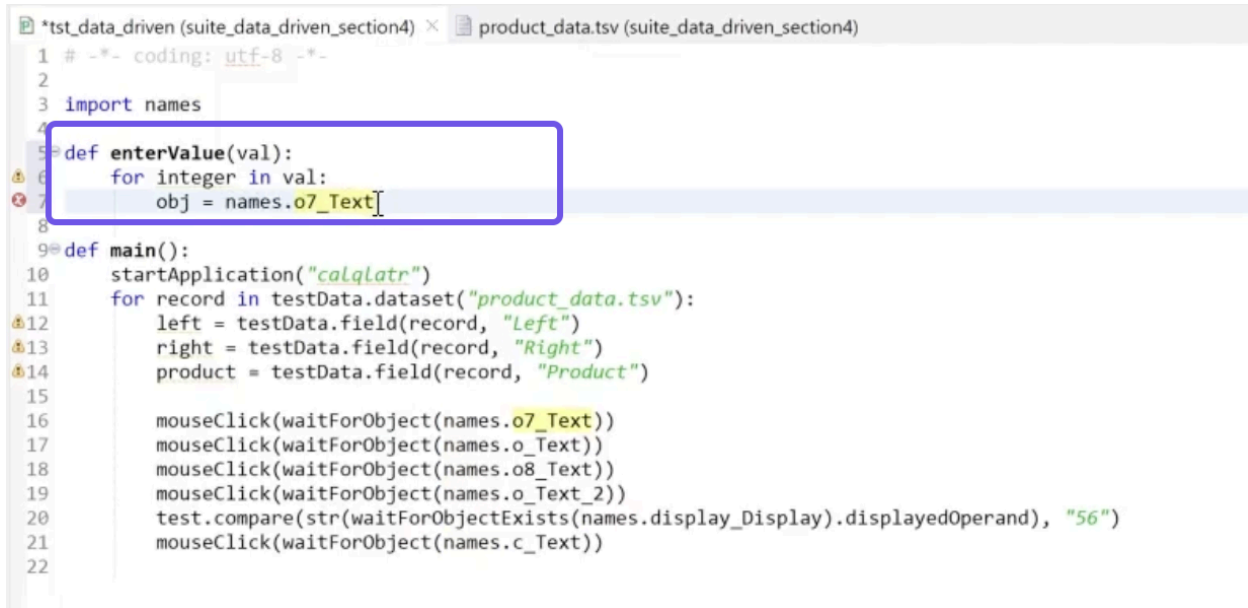
STEP 05 = Exploring the changes made to the original code



Our original code was been added to a "for" loop that is able to navigate through each cell of our table-based data and save the information from the table in the appropriate variables. Our left, right, and product data was saved in variables for each iteration of the "for" loop, which executes

for as many rows as exist in the table, since the `testdata.dataset` function returns an array of records from our test data. For each column name, a variable was created to hold the value of the test data in the corresponding column with the `testdata.field` function. With this “for” loop created, we could access every cell of data from our test data table.

STEP 06 = Modifying the code



```
*tst_data_driven (suite_data_driven_section4) × product_data.tsv (suite_data_driven_section4)
1 # -*- coding: utf-8 -*-
2
3 import names
4
5 def enterValue(val):
6     for integer in val:
7         obj = names.o7_Text
8
9 def main():
10     startApplication("calqlatr")
11     for record in testdata.dataset("product_data.tsv"):
12         left = testdata.field(record, "Left")
13         right = testdata.field(record, "Right")
14         product = testdata.field(record, "Product")
15
16         mouseClick(waitForObject(names.o7_Text))
17         mouseClick(waitForObject(names.o_Text))
18         mouseClick(waitForObject(names.o8_Text))
19         mouseClick(waitForObject(names.o_Text_2))
20         test.compare(str(waitForObjectExists(names.display_Display).displayedOperand), "56")
21         mouseClick(waitForObject(names.c_Text))
22
```

This test script simply selected 7 times 8 each time the “for” loop iterated. We wanted to change this by changing the way the values in the calculator are entered. To do this, we created a function called **enter value**, which would be able to click the appropriate button on the calculator based on the input from our test data file. We created the definition for this function, which takes one argument called `val`. `Val` will be either the left or right value from our test data file.

def enterValue(val):

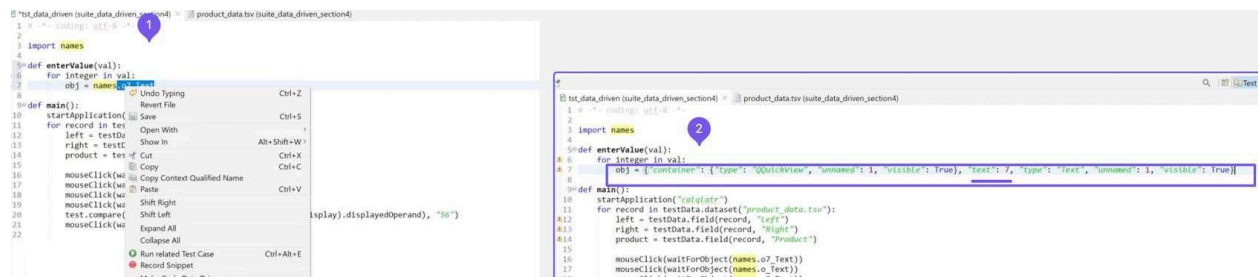
In this function, we used “`val`” to identify the real name for a button object in the calculator to be selected since, at this point, we did not have all the symbolic names stored in our object map. To account for multi-digit numbers, we iterated through `val` in a “for” loop.

for integer in val:

The next step was to identify the correct real name of the object to be clicked. We based this off on one of the buttons we have already clicked in the calculator. We set this up by copying the symbolic name from our original mouse click and saving it in a variable.

obj = names.o7_Text

STEP 07 = Converting the symbolic name to real name



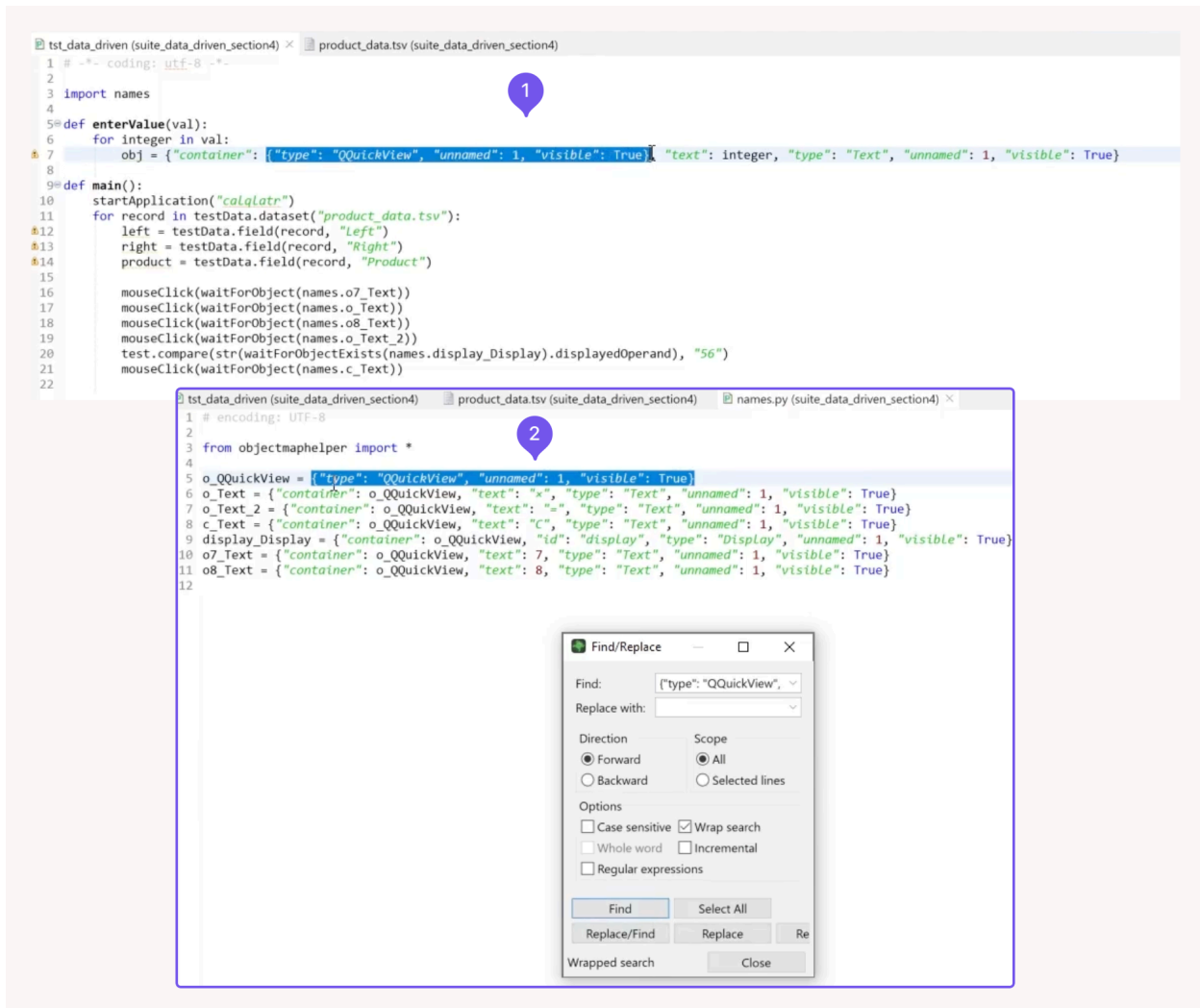
1. We converted the symbolic name to a real name in the Squish IDE by right-clicking and selecting “Convert to Real Name.”

2. Upon clicking this button, we saw the real name for the “7” button appeared in the code:
`obj = {"container": {"type": "QQuickView", "unnamed": 1, "visible": True}, "text": 7, "type": "Text", "unnamed": 1, "visible": True}`

The other buttons in the calculator shared the same properties as the “7” button, except for the text value. So, we used the “integer” value from our “for” loop instead of creating the proper real name.

`obj = {"container": {"type": "QQuickView", "unnamed": 1, "visible": True}, "text": integer, "type": "Text", "unnamed": 1, "visible": True}`

STEP 08 = Simplifying the real name, part 1



The real name was a bit long and contained a real name within another real name, so we simplified the name a bit by using the symbolic name of the container property's object.

1. We highlighted the brackets within the container property, copied the text.
2. Next, we switched over to the names.py object map file and searched for this text with control "f." We saw there was a symbolic name associated with these properties. We copied that symbolic name to use in our script.

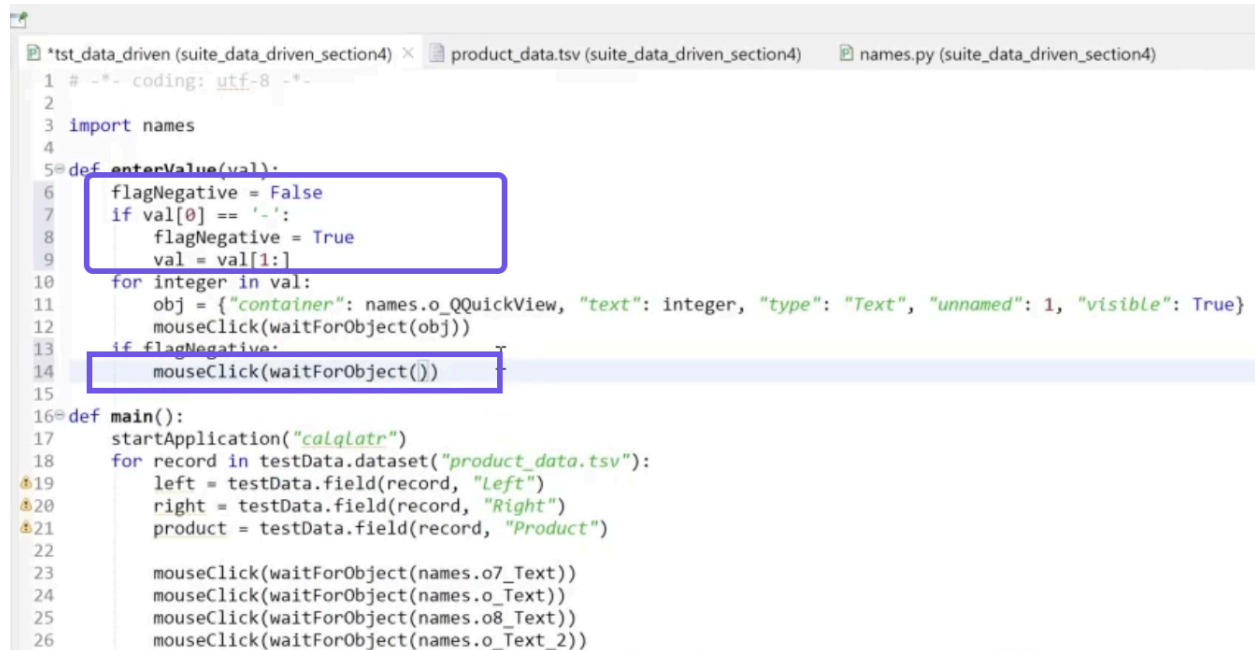
STEP 09 = Simplifying the real name, part 2



Back in our script, we highlighted the real name and replaced it with the symbolic name. We needed to add our names module before this symbolic name because it is an imported module.

```
obj = {"container": names.o_QQuickView, "text": integer, "type": "Text", "unnamed": 1, "visible": True}
```

STEP 10 = Modifying the enterValue function



```
1 # -*- coding: utf-8 -*-
2
3 import names
4
5 def enterValue(val):
6     flagNegative = False
7     if val[0] == '-':
8         flagNegative = True
9         val = val[1:]
10     for integer in val:
11         obj = {"container": names.o_QQuickView, "text": integer, "type": "Text", "unnamed": 1, "visible": True}
12         mouseClick(waitForObject(obj))
13         if flagNegative:
14             mouseClick(waitForObject())
15
16 def main():
17     startApplication("calcitr")
18     for record in testData.dataset("product_data.tsv"):
19         left = testData.field(record, "Left")
20         right = testData.field(record, "Right")
21         product = testData.field(record, "Product")
22
23         mouseClick(waitForObject(names.o7_Text))
24         mouseClick(waitForObject(names.o_Text))
25         mouseClick(waitForObject(names.o8_Text))
26         mouseClick(waitForObject(names.o_Text_2))
```

Next, we added a mouseClick action on the appropriate object:

mouseClick(waitForObject(obj))

The last thing we wanted to do was have our enterValue function account for negative numbers. So, we added a condition at the top to add a flag if any negative number is detected in our test data input. If the first digit in our function argument is negative, we would need to set a flag to true. So, added the following code:

flagNegative = False

if val[0] == '-':

flagNegative = True

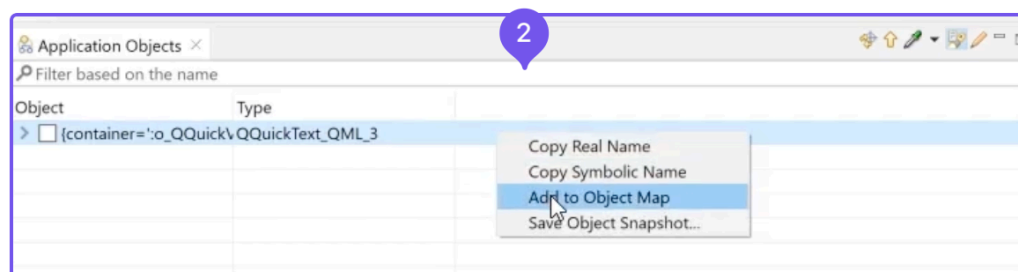
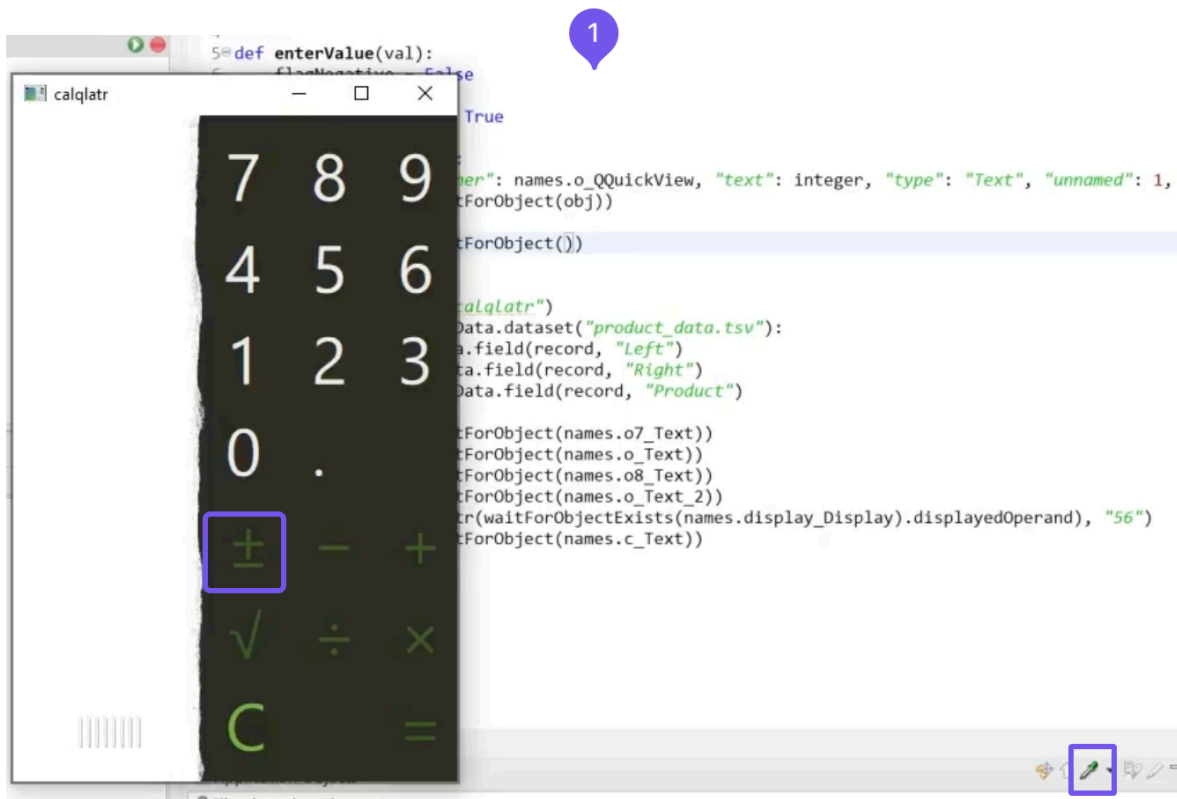
val = val[1:]

At the end of the function, we just added a condition that said, "if flag negative equals true, select the negative sign on the calculator."

if flagNegative:

mouseClick(waitForObject())

STEP 11 = Adding the negative sign to the object map, part 1



At this point, we did not have the negative sign in our object map, so as a next step, we added it.

1. To do this, we launched the AUT. With the AUT running, we selected the picker tool from the Application Objects view and selected the plus/minus button. This showed the object in our Application Objects view.
2. From here, we simply right-clicked and selected "Add to Object map."

STEP 12 = Adding the negative sign to the object map, part 2

1 # encoding: UTF-8

2

3 from objectmaphelper import *

4

5 o_QQuickView = {"type": "QQuickView", "unnamed": 1, "visible": True}

6 o_Text = {"container": o_QQuickView, "text": "x", "type": "Text", "unnamed": 1, "visible": True}

7 o_Text_2 = {"container": o_QQuickView, "text": "=", "type": "Text", "unnamed": 1, "visible": True}

8 c_Text = {"container": o_QQuickView, "text": "C", "type": "Text", "unnamed": 1, "visible": True}

9 display_Display = {"container": o_QQuickView, "id": "display", "type": "Display", "unnamed": 1, "visible": True}

10 o7_Text = {"container": o_QQuickView, "text": 7, "type": "Text", "unnamed": 1, "visible": True}

11 o8_Text = {"container": o_QQuickView, "text": 8, "type": "Text", "unnamed": 1, "visible": True}

12 **o_Text_3** = {"container": o_QQuickView, "text": "±", "type": "Text", "unnamed": 1, "visible": True}

13

1 # -*- coding: utf-8 -*-

2

3 import names

4

5 def enterValue(val):

6 flagNegative = False

7 if val[0] == '-':

8 flagNegative = True

9 val = val[1:]

10 for integer in val:

11 obj = {"container": names.o_QQuickView, "text": integer, "type": "Text", "unnamed": 1, "visible": True}

12 mouseClicked(waitForObject(obj))

13 if flagNegative:

14 mouseClicked(waitForObject(names.**o_Text_3**))

15

16 def main():

17 startApplication("calcLatr")

18 for record in testData.dataset("product_data.tsv"):

19 left = testData.field(record, "Left")

20 right = testData.field(record, "Right")

21 product = testData.field(record, "Product")

22

1. We stopped the application and checked our object map in names.py to see the symbolic name for the object. We saw it there, so we copied that symbolic name and used it in our script.

2. We went back to the script and finished our mouseClicked function:

mouseClick(waitForObject(names.o_Text_3))

STEP 13 = Making last modifications to the code

```
tst_data_driven (suite_data_driven_section4) × product_data.tsv (suite_data_driven_section4) names.py (suite_data_driven_section4)
1 # -*- coding: utf-8 -*-
2
3 import names
4
5 def enterValue(val):
6     flagNegative = False
7     if val[0] == '-':
8         flagNegative = True
9         val = val[1:]
10    for integer in val:
11        obj = {"container": names.o_QQuickView, "text": integer, "type": "Text", "unnamed": 1, "visible": True}
12        mouseClicked(waitForObject(obj))
13    if flagNegative:
14        mouseClicked(waitForObject(names.o_Text_3))
15
16 def main():
17     startApplication("calculator")
18     for record in testData.dataset("product_data.tsv"):
19         left = testData.field(record, "Left")
20         right = testData.field(record, "Right")
21         product = testData.field(record, "Product")
22
23         enterValue(left)
24         mouseClicked(waitForObject(names.o_Text))
25         enterValue(right)
26         mouseClicked(waitForObject(names.o_Text_2))
27         test.compare(str(waitForObjectExists(names.display_Display).displayedOperand), product)
28         mouseClicked(waitForObject(names.c_Text))
29
```

Lastly, we replaced the original mouseClicked functions in our data-driven code with the enterValue function, which is able to handle any input data.

enterValue(left)

mouseClicked(waitForObject(names.o_Text))

enterValue(right)

Then, we needed to make sure our comparison was not using any hard-coded values but rather the product value from our test data. To do this, we changed the second argument in the compare function to our product variable.

test.compare(str(waitForObjectExists(names.display_Display).displayedOperand), product)

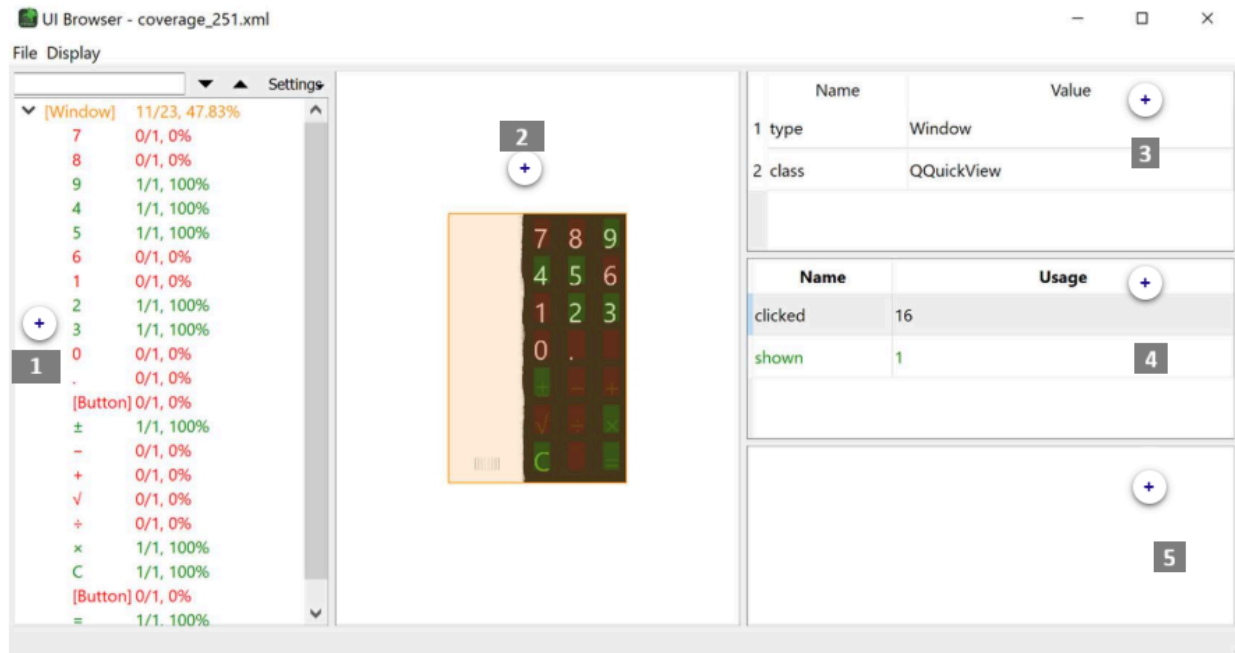
GUI Coverage

[GUI coverage](#) is often used with data-driven testing. GUI coverage is a form of program verification that assesses the number of GUI elements interacted with when running a test suite. When running tests with GUI coverage, information about interactions with specific elements, their properties, and the events themselves become available in the test results summary. Please note GUI coverage is in the Technical Preview state and can sometimes create issues with specific widgets. GUI coverage is “off” by default because it is not fully supported and has a performance impact when running tests. GUI coverage is available only when using Squish for Qt and should only be turned on when needed.

GUI coverage can be enabled by setting the environment variable SQUISH_GUI_COVERAGE=1. The environment variable can be set in Test Suite Settings > AUT > AUT Environment. The squishserver must be restarted, or a new test should be run in the Squish IDE to see the changes. With GUI coverage enabled, XML Coverage results are written for each test case. These can be viewed in the Test Results summary in the Squish IDE. GUI Coverage can be viewed by right-clicking GUI Coverage Results and selecting View Coverage Results, which will show a [UI Browser](#).

Test Results ✕ Test Summary Runner/Server Log Test Description	
Result	Message
▼ TestSuite	suite_MyTestSuiteDataDriven
▼ TestCase	tst_case_datadriven
> Pass	Comparison
> Pass	Comparison
> Pass	Comparison
> Pass	Comparison
> Pass	Comparison
GUI Coverage Result	GUI Coverage Result

The UI Browser contains 5 parts (click the + buttons on the picture to read more):



1	Element Tree	A hierarchy of GUI elements and their coverage. This displays information about all the elements and how many times they were interacted with, as well as what percentage of elements in the GUI were interacted with.
2	Screenshot	Display of the element selected in the Element Tree. Elements that were interacted with are highlighted in green, and unused elements are highlighted in red.
3	Element Properties	Static properties of the selected GUI element class. This displays the name of static properties of the selected element and their values.
4	Usage	List of the events registered for the selected element and its sub elements. This tracks the different types of events that were registered when interacting with the GUI as well as the number of times those events occurred.
5	Style Check	If the design of an element does not follow certain style rules, warnings are displayed in this window.

Summary

Data-driven testing is a software testing methodology

In data-driven testing, test input and output data is not written in the test script but stored in an external file that is loaded at runtime. This allows for the separation of test data and test script code when testing an AUT. There are many advantages to using data-driven testing, including the ability to modify test values in the test data, rather than the test script. This means it is not required to include hard-coded values in the script itself, since data can be stored in the separate test data file or database.

Squish supports Excel (.xlsx), CSV, and TSV table-based data sources

When creating data-driven tests, test data stored in a table format can be used to store the input and expected output. Squish supports the following table-based data sources: Excel (.xlsx), CSV, and TSV (.csv and .tsv files are assumed to be in the Unicode UTF-8 encoding).

Data tables can be created in Squish IDE or externally

It is possible to create data tables using the Squish table editor by accessing Test [Case/Suite] Resources > Test Data > New Test Resource. Tables can also be created externally and imported into the Squish IDE using the Import Squish Resource dialog accessed at File > Import Test Resource.

Locate the test data files by using findFile(where, filename) function

The return value of the findFile(where, filename) function can be used to access test data within a test script. This function takes two arguments and returns the path of a given filename. When using data stored in the Test [Case/Suite] Resources, the first argument of the findFile(where, filename) function can be the keyword “testdata”, which will cause findFile() to look first in the “testdata” directory of the test case for the data file. If findFile() does not locate a test data file in this directory, it

will look in the “testdata” directory of the test suite. Lastly, findFile() will try looking in the testcase’s directory, and if no file is found it will return a LookupError.

The Squish IDE can convert code to data-driven code

Existing code can be made data-driven by selecting the appropriate lines, right-clicking, and selecting “Make Code Data Driven”. The “Make Code Data Driven” option is only available if there is a test data file available to the test script.

GUI coverage is often used with data-driven testing.

GUI coverage is a form of program verification that assesses the number of GUI elements interacted with when running a test suite. When running tests with GUI coverage, information about interactions with specific elements, their properties, and the events themselves become available in the test results summary.

Please note GUI coverage is in the Technical Preview state and can sometimes create issues with specific widgets. GUI coverage is “off” by default because it is not fully supported and has a performance impact when running tests. GUI coverage is available only when using Squish for Qt and should only be turned on when needed.